Date: / /20

* Call By Value.

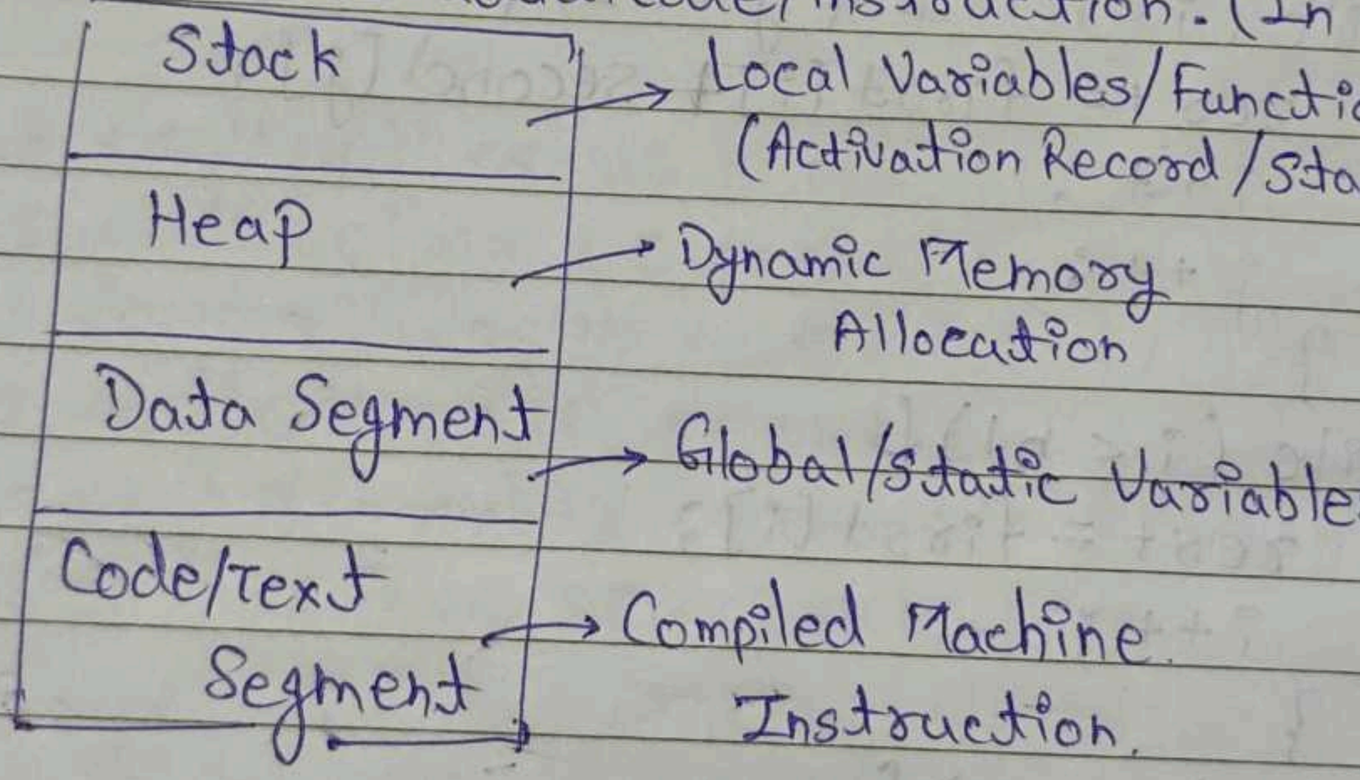
```
int main()
{
    int a=10;
//10  cout<<"Old a"<<a<<endl;
    increment(a);
//10  cout<<"New a"<<a<<endl;
}
```

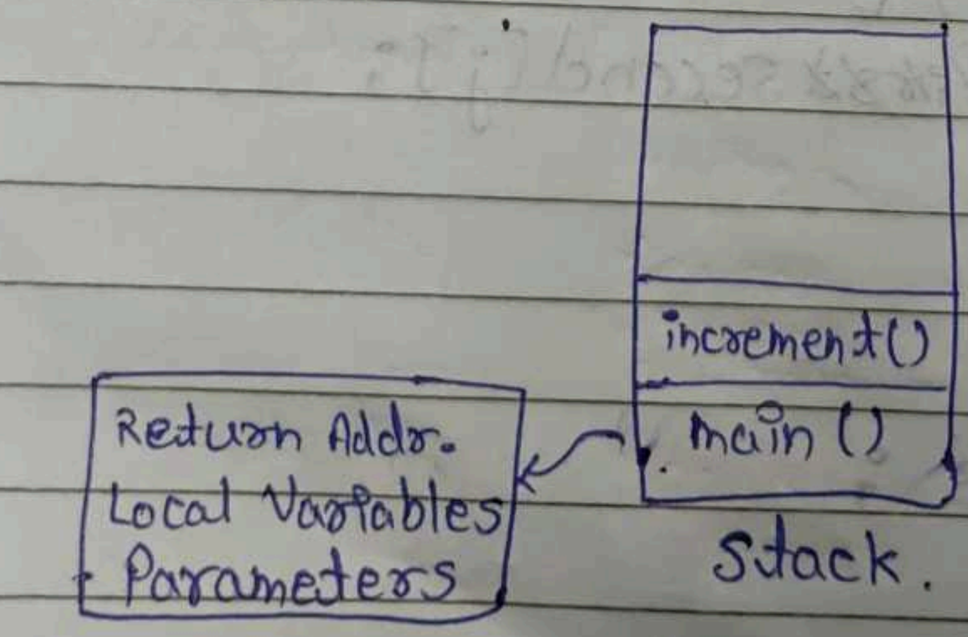
```
void increment(int a)   //10
{
    a=a+1;  //11
    cout<<a<<endl;
}                       //11
```

a | 10 | 100

a | 11 | 200

OS gives memory to the program. Compiler decides where to store the data/code/instruction. (In the RAM)





Date: / /20

* Type Qualifiers

The keywords,
- That modifies the properties of datatypes that influence how the variables/functions will behave.

→ const ↝ cannot modify value of variable
→ volatile
→ mutable.

```
const int a;
  a = 10;
    X
```

```
const int a=10;
```
↳ using this can help manage the scope.
- gets memory at

```
# define a 10;
```
↳ gets memory at

```
void inc(int a) const{
    a=a+1;
}.
```
↳ non-member functions cannot be 'const'

```
class A {
public:
    int a;
    int b;
    void inc(int) const {
        a=a+1;
        b=b+1;
    }
}
```
↳ here, the data member's can't be updated in the const. function.

To do it the data member need to be made mutable.

```
{ int a;
  mutable int b;
    a=a+1   X
    b=b+1   ✓
}
```

VIRAG

```
#define a 10
int main (){
    const int a=10;
    inc(a);
    cout << a <<endl;
}
void inc(int a){
    a = a+1;
}
```

① Compile Time Error
② 10
③ Run Time Error

→ o/p: 10.

volatile :— don't optimize it as the data/fn. may change by some h/w interrupt, signal, etc.

```
int main (){
    volatile int a = 10;
}.
```

* Storage Classes. Describe the characteristics of variable & function.

Keyword	Lifetime	Visibility	Initial Value.
Auto.	auto	Scope	Local	Garbage.
Extern	extern	Entire Program	Global	zero (0)
Static	static	Scope Entire Program	Local	0.
Register	register	Scope	Local	Garbage.
Mutable.	mutable.	Class	Local	Garbage

```
int main () {
    count ();
    count ();
    count ();
}
```

```
void count () {
    static int a = 0;
    a = a + 1;
    cout << "function called" << a;
}
```

Q. static for non-member function ?

↳ ~~extern~~ prevent external linkage of functions

array of pointer
pointer of array.
reference.

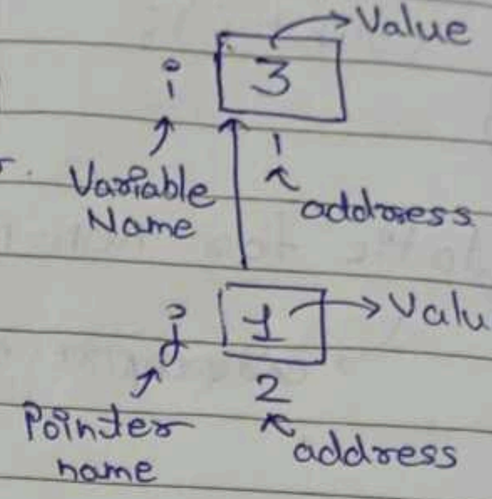
Date: 5/3/2026

MON TUE WED THU FRI SAT SUN

POINTERS

```
int main () {
    int i = 3;
    cout << &i << endl; //1
}
```

↳ addr. of operator



Pointer ⟶ `int * j = &i;`

↳ value at address

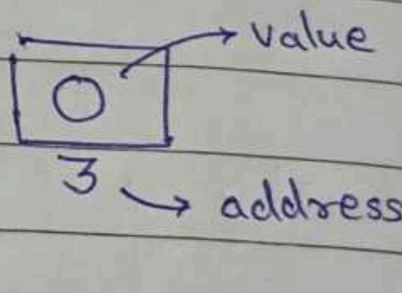
Pointers are special variable that are used to store address of another variable.

`int * m = 0;` (Equivalent to) `int * m = NULL;`

↳ NULL pointer

`* m = 10;`

↳ undefined behaviour
- crash / fault.
- some compilers can give compile-time error as 'trying to initialized a uninitialized value'




```
int main () {
    int * m = 10;
}
```

↳ gives error.
tries to point at a uninitialized memory location.

Write a integer pointer,

`int * p = NULL;`

(or)

`int * p;`

VIRAG

Date: / /20

— Reference

```
int main(){
    int p=10;
    int &ref=p;
}
```

p → |10|
ref →

```
int main(){
    int &ref;
    ref=p;
}
```

⇒ This is not allowed.

Q. Why can't we reinitialize the reference.

① Array of 5 integers
② Pointer to an integer.
③ Pointer to an array of 5 integer.
④ Array of 5 pointer to integer.

① → int arr[5]; → {1,2,3,4,5}
② → int *p = NULL;
③ → int (*p)[5]; p → |1|2|3|4|5|
④ → int *p[5];

→ {&p[0], &p[1], &p[2], &p[3], &p[4]}.
→ Array of pointer integers.

WAP to create a pointer to an array of 5 integers.
→ Syntax ?
→ Where the pointer will be pointing.
→ How the pointer will traverse.
→ How to access/print elements.
→ How to send the pointer to function ?

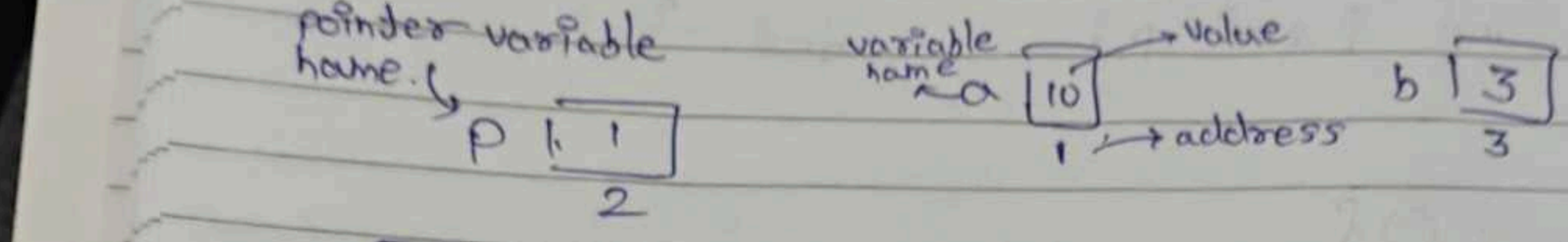
Q. IS int* p = &arr allowed and where does it point ?

Date: 6/3/2026

MON TUE WED THU FRI SAT SUN

(*) Pointer to an integer.

```
int a=10, b=3;
int *p= &a;
```




```
p1 [1]
p1 [NULL]
p1 [3]
```

```
int *p1 = &a;
p1 = NULL   (make the pointer p1 point to nothing)
p1 = &b;
```

```
int &ref = a;
ref = b;   X
```

(A reference cannot be re-initialized)

↳ A ref is not stored in memory, so one cannot reinitialize it.

```
int &ref;
ref = a;
```

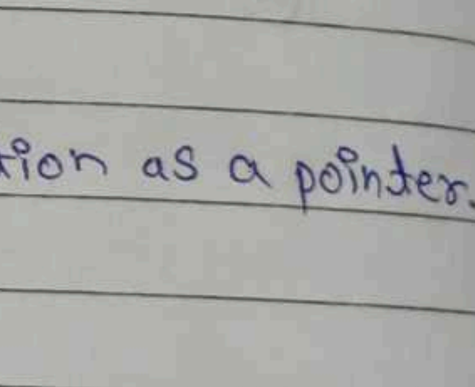
} Can't be done. Reference should be initialized at the time of declaration as it is a alias.

— A referenee does not have the information as a pointer.

(*) Pointer to array of 3 integer.

```
int arr[3] = {1,2,3};
int (*ptr)[3] = &arr;
for (int i=0; i<3; i++){
    cout << (*ptr)[i] << endl;
}
```

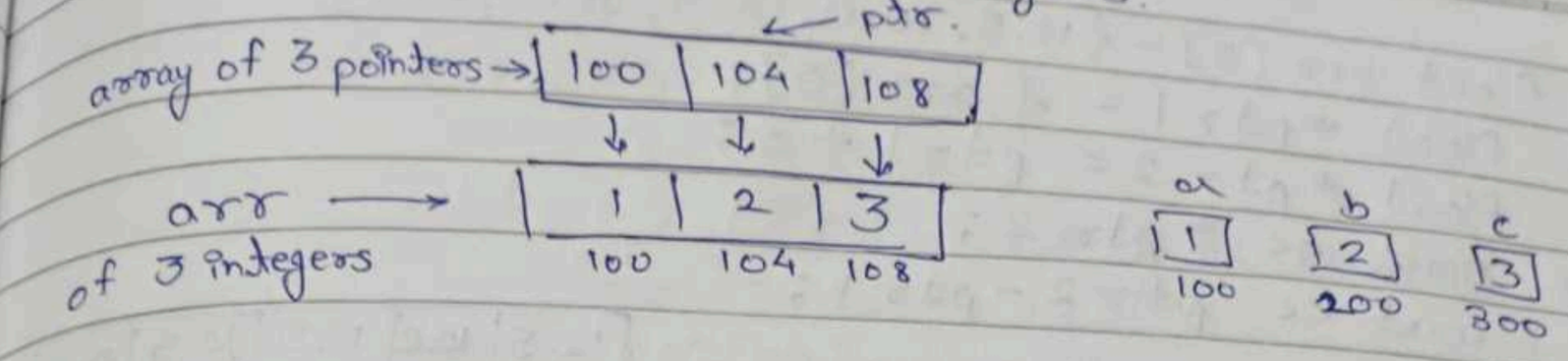
```
int *p= &arr;
*(p+i)
```



VIRAG

Date: / /20

(*) Array of 3 pointer to integers.



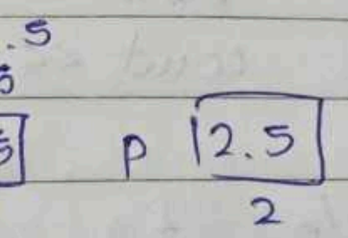

```
int a=1,b=2,c=3;
int *p[3] = {&a,&b,&c};

for(int i=0;i<3;i++){
    cout<<*p[i]<<endl;
}
```

(or)

```
cout<<*(*p+i)<<endl;
```

What will be the o/p:

```
float f=10.5;
float p=2.5;
float *ptr=&f;
(*ptr)++;
*ptr=p;
cout<<*ptr<<f<<p;
```



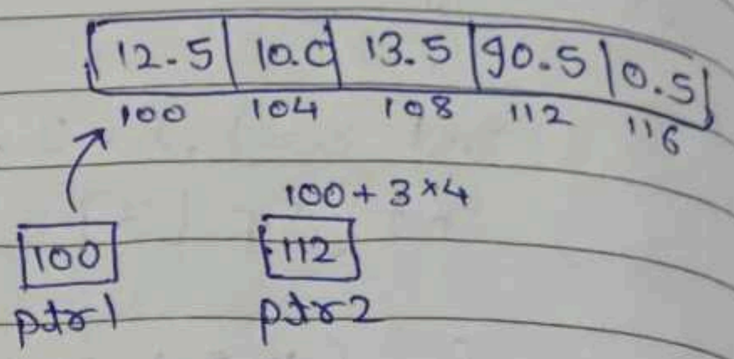
o/p:- 2.5 2.5 2.5.

```
void changeSign(int *p){
    *p=(*p)*-1;
}

int main(){
    int a=10;
    change-Sign(&a);
    cout<<a<<endl;
}
```

Date: / /20

MON TUE WED THU FRI SAT SUN

```
float arr[5] = {12.5, 10.0, 13.5, 90.5, 0.5}
float *ptr1 = &arr[0];
float *ptr2 = ptr1 + 3;
cout << *ptr2;
cout << ptr2 - ptr1;

// 90.5
// 112 - 100 = 12 = 12/4 = 3
```




```
char *ptr = NULL;
char str[] = "abcdefg";
     ptr = str;
     ptr += 5;
     cout << ptr;
```

o/p :- fg.

In a string, if address of a character is printed, then the pointer instead of address prints the string from pointer till '\0' character in string.

```
char st[] = "ABCD";
for(int i=0; st[i] != '\0'; i++){
    cout << st[i] << *(st) + i << *(i + st) << i[st] << endl;
}
```

```
int a=5, b=10, c=15;
int *arr[] = {&a, &b, &c};
 cout << *arr[1] << endl
 cout << arr[1] << endl.
```

Date: / /20

st[i] is same as i[st]

*(st)+i ⇒ *(st)+i

for i=0,
*(st)+0 = A+0 = 65+0 = 65.

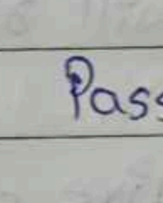
for i=1,
*(st)+1 = A+1 = 65+1 = 66.

*(i+st) ⇒ for i=0,
*(0+A) = *(0+65) = A.

for i=1,
*(1+A) = *(1+65) = B.

Pass by reference

```
void modify (int &a){
    a = a+1;
}

int main(){
    int a=1;
    modify(a);
    cout << a << endl;
}
```



Pass by address v/s Pass by reference

Real-life ex.
LL, Trees, (Data Struc

```
int add()
{
    int a=10;
    int &ref=a;
    return ref;
}
```

returning a ref. is not a good practice viz. bad practice.

Date 11/3/2026

Compiling & Linking

Source Code — .cpp
↓
Preprocessor — .i (clean expanded code)
↓
Compiler — .s
↓
Assembler — .obj
↓
Linker — .exe
↓
Loader

→ A program under execution is called process.

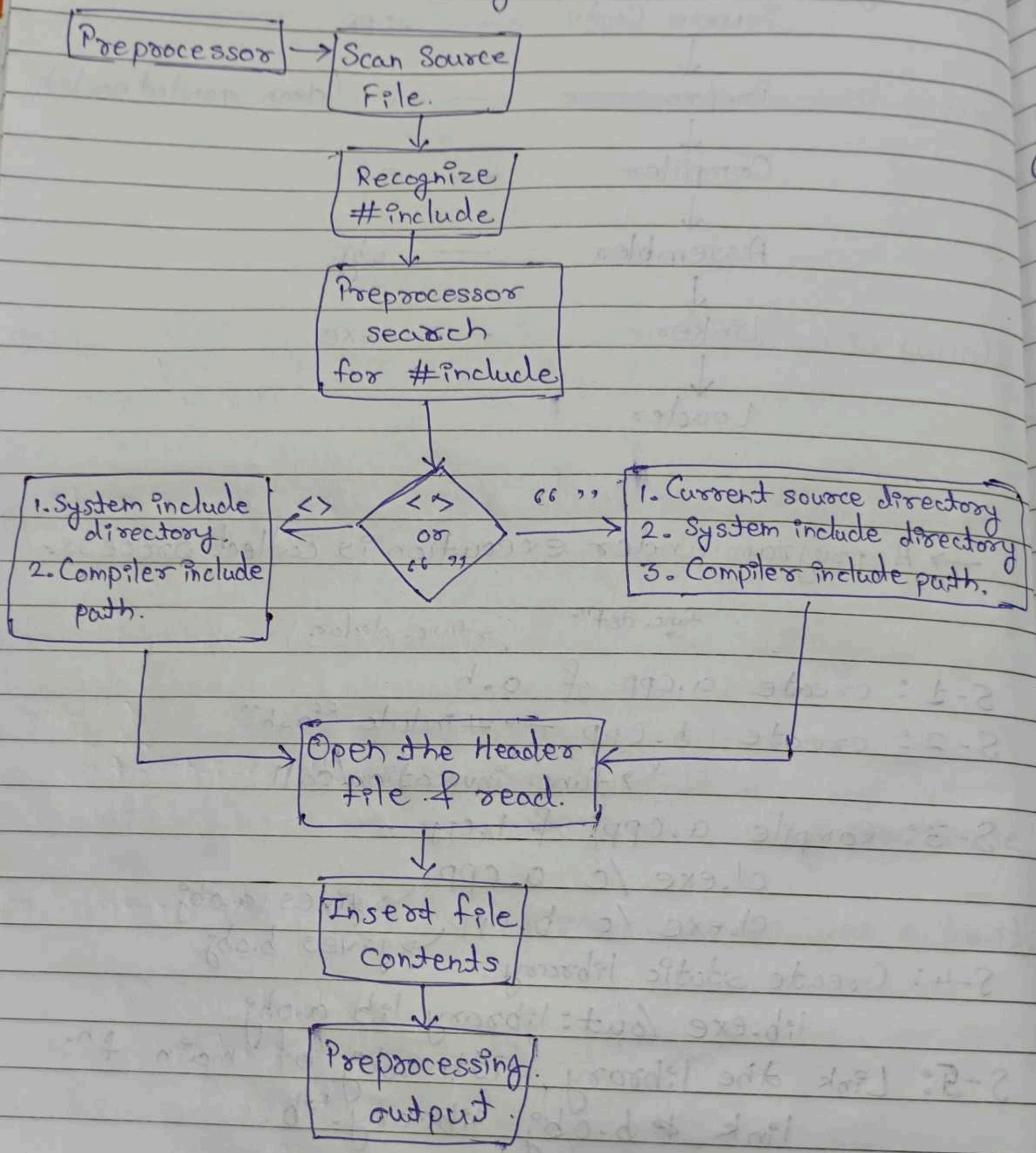
func. def^n.
↗ func. declar.

S-1: create a.cpp & a.h
S-2: create b.cpp → #include "a.h"
↳ func. invocation/call.
S-3: Compile a.cpp & b.cpp
cl.exe /c a.cpp → gives a.obj.
cl.exe /c b.cpp → gives b.obj
S-4: Create static library.
lib.exe /out:library.lib a.obj.
S-5: Link the library with .obj of main fn.
link b.obj library.lib.
↳ b.exe file.
S-6: Result
b.exe → Output

Date: / /20

MON TUE WED THU FRI SAT SUN

Internal working of #include.



* Guards for multiple includes.

Include Guards or backend to prevent multiple includes & keep track of if a library is already included.

* Encap

* Ab

THU FRI SAT SUN

Date: 12/3/2026

OOPS (Object Oriented Programming)

(*) Encapsulation
- wrapping up a data in a class.
- creating a capsule.
- used in data hiding. using,
  Private
  Protected
  Public
  / Classes & objects.
  ↑
  Implemented using

(*) Abstraction.
- Hiding complex implementation
- Sharing necessary details/features.
- Implemented using,
  Abstract Class / Pure Virtual Functions.

Class - holds data members & member functions.
- blueprint of a object.
- User Defined data-type.

Object - instance of a class.

The Only the data members are stored in. object. It does not have member fn. with it, it only has the address of the fn.

Q. Return different types of data using a getter in a class?

directory
directory
e path.

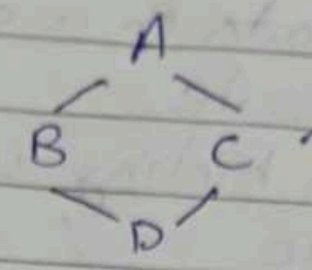
tiple
is

VIRAG

Date: / /20

✱ Inheritance

- Single Inheritance
- Multilevel
- Multiple
- Hierarchical
- Hybrid



Diamond Problem.

```
class A
public & protected:
    i=10

class B
public: void f(){
    :
}

main()
  B b;
  cout << b.i << endl;
```

```
class A
protected:
    i=10

Class B
P
```

✱ Polymorphism.

Fn. Overloading.

- return type is not considered as signature of function.
- Only the number & type of arguments decide it.

Date: 13/3/2026

Operator Overloading

```
{
  complex obj(2,3);
  obj.print();
  complex obj1(4,5);
  complex obj2 = obj + obj1;
  obj2.print();
  return 0;
}
```

obj.operator+(obj1) → Internally obj.operator+(obj1)

+, -, *, /

Not possible for objects (User Defined Data Type)

⇒ We do operator overloading for it. (Compile Time Poly.)

```
Complex operator+ (Complex C1){
    Complex C2(0,0);
    C2.real = real + C1.real;
    C2.imaginary = imaginary + C1.imaginary;
    return C2;
}
```

(obj → this, obj1 → C1)

* Static Member Variables.

Initialization should be outside

```
int Player::count = 1

class Player{
  private:
    static int count;
  public:
    Player(){count++};
}

static void PrintCount(){
    cout << count;
}

int main(){
    Player P1;
    Player P2;
    Player P3;
}
```

P1.printcount()
P2.printcount()
P3.printcount()

actually works as Player::printcount()

↳ Because anyways we do not create the print count is not binded to object.

Date: / /20

static print count can't have non-static data member

```
private:
    static int count;
    int m = 10
```

```
printcount(){
    cout << count;
    cout << m;
}
```

```
Class student {
public: private:
        int i=10;
        int m=20;
public:
    void modify() const {
        i = 30;
        m = 40;
    }
};
```

```
int main(){
    student s1;
    s1.modify();
}
```

fn.
In const we can't modify the data member. To do that we, need to make the data member mutable.

```
mutable int m=20;
```

(*) Friend Function / Class.

Friend fn. from another class.

```
Class A {
private:
    int m;
protected:
    int n;
```

- stack unwinding

Date: / /20

(*) Method Overriding.

```
{
    Base B;
    B.display();
Derived D, D1;
    D.display();
Base* B1 = &D1;
    B1.display();
}
```

→ This should point to the derived class by our understanding. But when we use the display fn of derived class in this case, it will print from base class. To do the other way we need to make the display fn. virtual.

override keyword

- it is optional, we can use it in overridden fn. to prevent doing any mistakes in fn. definition, arguments, parameters, ----

⇒ final keyword.

with a fn. - can't be overridden.
with a class - can't be inherited.
with a variable - can't be changed.

Virtual enables runtime polymorphism by calling correct fn. via base class pointer or reference.

Q. Class C1 & C2 have data member a & b private respec. Write a code to get the sum of a & b.

→

```
class C1 {
    private:
        int a;
};
public:
    friend function_sum()

class C2 {
    private:
        int b;
};
public:
    friend function_sum()
```

C1 C2

```
function sum (int x, int y) {
    cout << x+y << endl;
}
```

correct code:

```
class C2;
class C1 {
    int a = 10;
public:
    friend void sum (C1, C2);
};

class C2 {
    int a = 10;
    friend void sum (C1, C2);
}

void sum (C1 x, C2 y) {
    cout << C1.x + C2.y << endl;
}

int main() {
    C1 a;
    C2 b;
    sum(a, b);
}
```

Ans:

1 - a class as a friend fn. of another class
- a friend fn. of a single class & mem. fn. of another class
- a friend fn. of both the class.

VIRAG

```
    private:
        int c = 0;
};
// Overload pre & post increment operator.

operator ++(Counter &t){

void operator ++ ( ){
        //pre-increment
        return ++c;
    }

void operator ++ (int){
        //post-increment
        return c++;
    }

int main( ){
    Counter c1;                    int i=1;
    ++c1;                          ++i;      // 2
    c1.display( );                 i++;      // 2
    c1++;                          cout<<i<<endl; //3
    c1.display( );
}
```

Q. Overload greater than (>) & lesser than (<) operator.

→

```
class A {
    int a=10;
};

class B {
    int b=20;
};
```

VIRAG

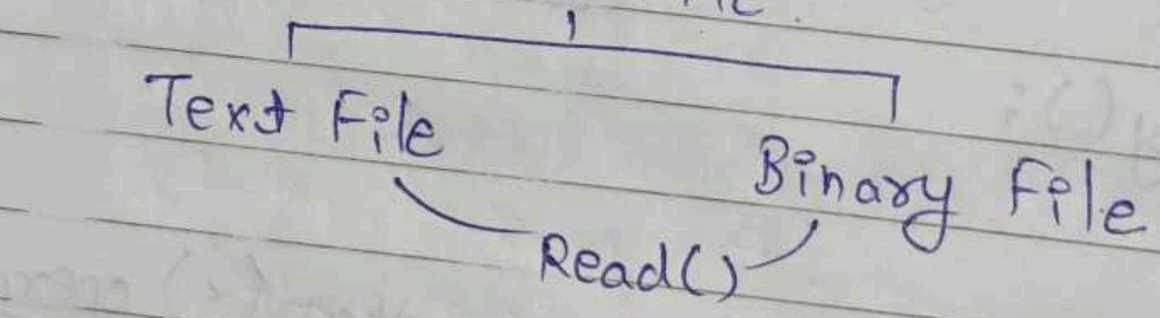
```
class C{
    int n;
};

C operator > (C c1){
    return n > c1.n;
}

C operator < (C c1){
    return n < c1.n;
}
```

```
int main(){
    C obj, obj1, obj2;

    obj.C obj(1);
    C obj1(4);
    C obj2(3);

    cout>> obj > obj1;
    cout>> obj1 > obj;
    cout>> obj2 < obj;
}
```

Q.

const data member fn :- We cannot modify the value of a data member ~~of the object~~ through which the the class; object is called.

Q.




```
Class File{
public:
    virtual void Read(){} = 0;
};

class TextFile: public File{
public:
    void Read(){ cout << "Text File" << endl};
```

```
class BinaryFile{
    public:
        void Read(){cout<<"Binary file"<<endl};
    }
```

Q. Convert hexadecimal to decimal.

→

```
int main(){
    int arr[6]={10,11,12,13,14,15,~~66~~};
string ~~int~~ hex;
    cin >> ~~hex~~; int decimal=0;
getline(cin, hex);
 for(int i=0;i<hex.length;i++){
    decimal+= Number(hex[i]*(16**(hex.length-1)));
    }
 ~~int~~ cout << decimal;
```

6A

$6 \times 16^1 + 16^0 \times 10$

Q. Given an array of 10 strings, sort them in order of dictionary.

```
for(int i=0;i<9;i++){
  for(int j=0;j<9-i;j++){
      if(arr[j]>arr[j+1]){
          string temp=arr[j];
          arr[j]=arr[j+1];
          arr[j+1]=~~arr~~ temp;
      }
  }
}
```

→ Bubble Sort

Date: / /20

Q.1. Sort 0-1-2 in an array

| 0 | 1 | 0 | 0 | 2 | 1 | 2 |

↳ | 0 | 0 | 0 | 1 | 1 | 2 | 2 |

```
int main () {
    int arr[7] = [0, 1, 0, 2, 0, 1, 2];
    int i=0, j = arr.length() - 1;
    for (int n = 0; n < arr.length(); n++) {
```

Q. Reverse a string.
I/p: 'Hello C++ Training'
o/p: 'Training C++ Hello'

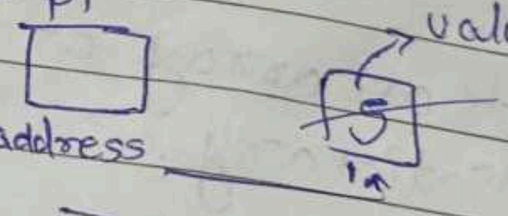
17-3-26

Dynamic Memory Allocation

Funct^n. — New / Delete.
User Data. Operator

```
int *pi = new int(5);
delete pi;
pi = NULLPTR;
```

pointer var. name → pi




```
int *pi = new int;
cout << *pi;
```
→ gives a garbage value.

```
int *pi = new int ();
cout << *pi;
```
→ initialized to 0.

→ Check for arrays. - what is initialized in the array elements?

e.

VIRAG

If a dynamic memory is allocated & the memory is not allocated, it will give an exception 'Bad Alloc' on heap

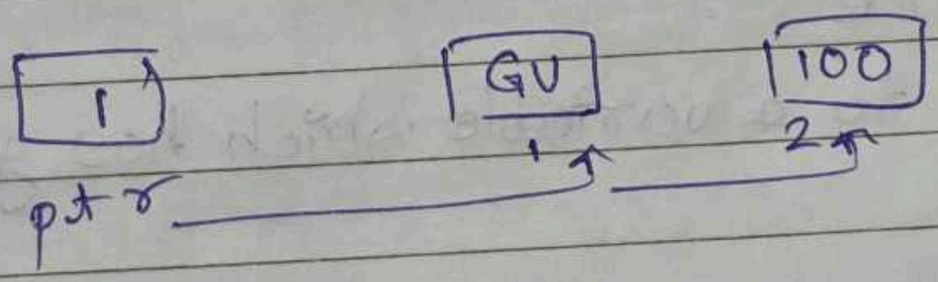
To prevent not throwing this exception we can do as below,

```
int *pi = new(nothrow) int [5];
```

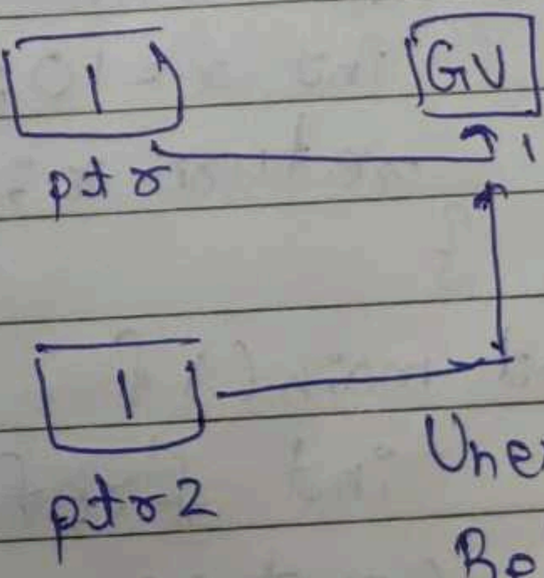
e.g.
```
int *pi = NULL;
delete pi;
```

e.g.
```
int *ptr = new int;
cout << ptr << *ptr <<endl;
```
↑ address ↑ garbage

e.g.
```
int *ptr = new int;
ptr++;
delete ptr;
```
→ Runtime Error
→ does not allow next memory addr. to be delete which is not related to our program.
Unauthorized memory deletion



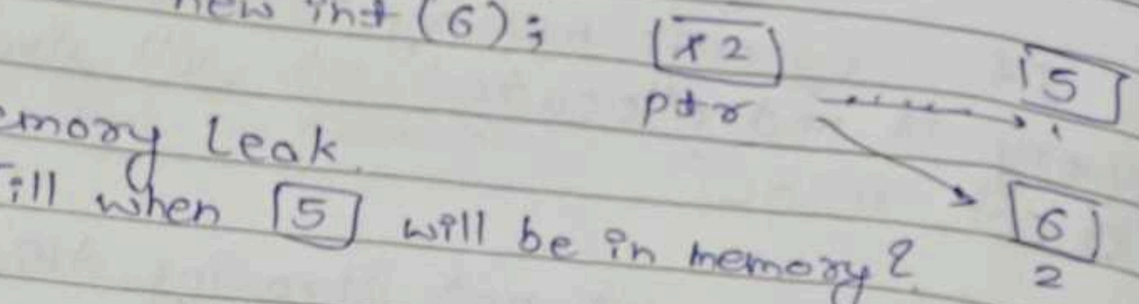
e.g.
```
int *ptr = new int;
int *ptr2 = ptr;
delete ptr;
delete ptr2;
```



→ Undefined output.
Unexpected Behaviour.

≈ Dangling Ptr. Concept.

e.g. int *ptr = new int(5);
ptr = new int(6);



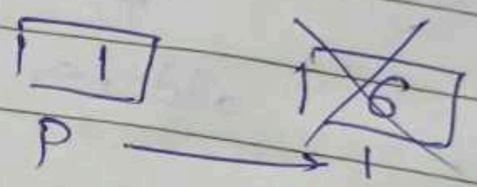
↳ Memory Leak

Q. Till when [5] will be in memory?

-[5] can't be accessed later in the program if it was not deleted by the programmer.

Q. Memory Leak
Q. Dangling Pointer & Dangling Reference with examples.
Q. Double Deletion.

e.g. int *p = new int(6);
delete p;
cout << *p << endl.



// Good practice
p = NULL;

Dangling Pointer.

* Dangling Reference
Ref. pointing to a variable which has gone out of scope.

```
int &fun(){
    int x = 10;
    return x;
}
int main(){
    int &ref = fun();   // Dangling Ref.
    cout << ref << endl;
}
```

Works in VS but actually not possible.

→ A static fn. is restricted to the file in which it is declared/defined.
- The fn. can't be used in other files

e.g.
```
int x = 10;
void *ptr = &x;        → can convert the pointer to respective data type
cout << *ptr << endl;  → does not point anything
cout << *(int*)ptr << endl;  → doesn't know what type of data is to be printed
```

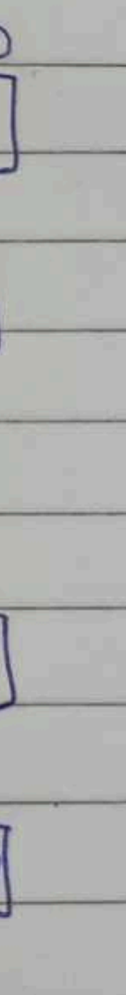
Void Pointer

⊛ Shallow Copy & Deep Copy.

- Shallow Copy.

```
class A {
    int a, b; int *p;          p = new int(5);
    A (int a1, int b1) { a = a1, b = b1 };
    void display() {
        cout << a << " " << b;
    }
    void modify() {
        a = 50;
        *p = 10
    }
};
int main() {
    A obj1(10, 20);
    A obj2 = obj1;
    obj1.modify();
    obj2.modify();
}
```

Internally,
dually obj2.a = obj1.a
obj2.b = obj1.b



// Copy Constructor

```
A (const A &a){
    - - - - -
}
```

→ The copy which we going to create should not be modifiable & the value's should be copied as it is. So 'const' is used.

→ ~~God~~ Good Practice to use const

A copy constr. can also be created as below,

```
A ( A a) {
    - - - -
}
```

e.g. Copy Constructor.

```
class A {
    int a, b;
    int *p = NULL;
};
public:
    A (int a1=0, int b1=0){
        a = a1;
        b = b1;
        p = new int(5);
    }
void modify (){
    a = 50;
    *p = 1000;
}
void display (){
    cout << a << " " << b << " " << *p;
}
A (const A &obj2){
    a = obj2.a;
    b = obj2.b;
    *p = new int (obj2.*p);
}
```

(OR)

```
*p = new int (*(obj2.p));
```

Q. Why reference in copy constructor (A & obj) → Infinite loop for copying the new created objects.

Date: / /20

```
int main(){
    A.obj1(10,20);
    A.obj2(obj1);
    obj2.display();   // a=10, b=20, *p=5
    obj1.modify();    // a=50, b=20, *p=1000
    obj2.display();   // a=10, b=20, *p=5
}
```

→ Call CC (Custom CC)
(or) A.obj2 = obj1

```
A obj3;
obj3 = obj1;
```

} It does not call custom CC.

— By default this creates a shallow copy.

— It requires overloaded assignment for deep copy.
— We need to overload assignment for it.

Overload Assigment Code:

```
A & operator=(const A & obj){
    a = obj.a;
    b = obj.b;
   *p = new int(*(obj.p));
    return *this;
}
```

(OR)

```
void operator=(const A & obj){
    a = obj.a;
    b = obj.b;
   *p = new int(*(obj.p));
}
```

⊛ Virtual Destructor.

```
class Base{
    int *p=new int();
public:
    Base(){
        *p=10;
    }  cout<<"Base CTQ"<<endl;
    ~Base(){
        delete p;
    }  cout<<"Base DTQ"<<endl;
};
```

There is no virtual constructor.

```
class D:public Base{
    int *d = new int();
public:
    D(){
        *d=20;
    }  cout<<"Derived CTQ"<<endl;
    ~D(){
        delete d;
    }  cout<<"Derived DTQ"<<endl;
};
```

```
int main(){
    Base *p=new D();  //Base class pointer, pointing to derived class.
    delete p;
    return 0;
}
```

→ In such kind of cases the Derived DTQ is not called. It is from the virtual Table resolving concept. To avoid memory leak & call Derived DTQ we need to make Base DTQ as virtual.

o/p:- Base CTQ
Derived CTQ
Base DTQ

```
virtual ~Base{
    delete p;
    cout<<"Derived DTQ"<<endl;
};
```

o/p:- Base CTQ
Derived CTQ
Derived DTQ
Base CTQ.

Date: / /20

ⓐ Function Pointer.

↳ holds address of function

```
int f  cout << f << endl
       cout << *f << endl;
```
↑ returns address of function

```
int f (int a, int b) {
    - - - -
}
```

(OR) `int (*fp)(int,int) = f;`

```
int (*fp)(int,int) = &f;
```

↑ return type of f^n.
↑ Name of pointer.
↑ Arguments data type of f^n.

```
fp = &f;
fp(1,1);
```

e.g. `int multiply(int,int){ }`

```
int (*p)(int,int) = multiply;
p(2,3);
```

(OR) `auto p = multiply;`

If we want to use a f^n. as a argument we can use f^n. pointer. Also if we want to return a f^n., we need to do that.

Q. a=10, b=20;

```
int mul() { return a*b;};
int sub() { return a-b;}
int add() { return a+b;}

void callfunc (int (*fp)) {
        return cout << fp << end;
}

int main(){
    int a;
    cin >> a;
    if (a==1) {
        // Call multiply
        call (*mul())
    }
else if (a==2) {
    // call sub
        call (sub())
}
else if (a==3) {
    // call add
        call (add())
}
```

Q. class A {
public:
int func (int a, int b) { ---- }
}

// Create a function pointer for func() :

Date: / /20

→ int main() {
int (A::*ptr)(int,int) = &A::func;
A a;
cout << (a.*ptr)(1,2);

Q. Create an array of function pointers.

18-3-26

class A {
private:
int m;
int (*p)[5]; pointer to integer array of size 5

//Implement copy constructor & assignment operator.

A(A &obj){
m = obj.m;
*p
A(int m){
this->m = m;
p = new int
A(){
m = 10;
int arr[5] = {1,2,3,4,5};
p = &arr;
}const
A(A &obj){
m = obj.m;
p =

int main(){
A a;
A b = a;
A a1;
a1 = a;
}

VIRAG

```
class A {
private:
    int m;
    // int (*p)[5];
    int *p;
public:
    A() {
        m = 10;
        for(int i=0; i<5; i++) {
            p[i] = i+1;
        }
    }
    A(const A &obj) {
        m = obj.m;
        p = new int[5];
        for(int i=0; i<5; i++) {
            p[i] = obj.p[i];
        }
    }
};
```

Type: int (*)[5]

pointer to an array of integers

```
int main() {
    A a;
    A b = a;
    A a1;
    a1 = a;
```

```
int (*p)[5] = new int[5];
```

Error: We are trying to assign value of type int* to a type int (*)[5]

↳ pointer to an array of 5 integer

↳ dynamically allocate space for 5 integers.

new int[5] returns a pointer of the related type here int i.e. int*.

// Copy Constructor.

- Check whether the this & given object are not same. If same do not process further copying.

object

```
A a;
a = a;
```

```
CC {
    if (this != obj) {
        return this.
    }
};
```

```
main(){
A a;
A b=a;
A a1;
a1=a;
```

Date: / /20

// Ope Assignment Overloading

Should return the object, if void we cannot perform optional chaining.

viz. a=b=c;

↳ for this to work we should return the respective object of that class.

Virtual Table And Virtual Pointer.

The compiler, when sees virtual func it creates a static array.

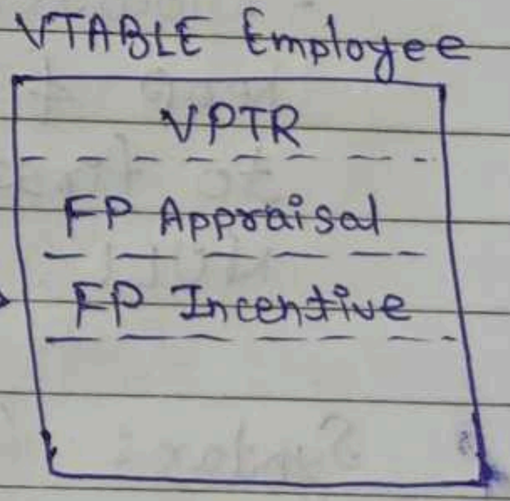
It creates the vtable for that class as well as it's child classes.

VTABLE has function pointer's pointing to the functions in the class.

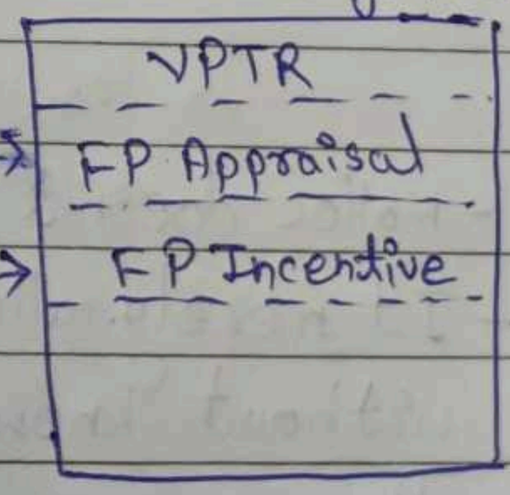
It only has the virtual functions.

* This process of VTABLE happens at runtime.

```
class Employee{
public:
    virtual void Appr.(){}
    virtual void Incen.(){}
}
```




```
class Eng. : public Emp.{
    public:
        void Appr.(){}
        void Incen.(){}
}
```



Vtable / Vptr
OOP's concepts
const keyword
Abstract Classes (Method overloading)

VPTR is used to reach VTABLE. It is the part of the object created.
VPTR is inherited in every VTABLE.

VTABLE does not contain, any other functions than virtual.

④ malloc, calloc & realloc.

- Available in 'stdlib.h' header file, also <cstdlib>

* malloc.
- used to dynamically allocate a single large block of contiguous memory according to the size specified.

ptr = (datatype *) malloc (SIZE);

e.g.
int *p;
p = (int*) malloc (20);

- allocates memory according to size specified in heap & on success returns a pointer pointing to first byte of allocated memory else returns NULL.

Syntax: (void*) malloc (size_t size)
↘ unsigned int

- malloc doesn't have an idea of what it is pointing to.
- It merely allocates memory requested by the user without knowing the type of data to be stored inside the memory.

VIRAG

Date: / /20

- The void pointer is later typecasted to an appropriate type.

```
int *ptr = (int*) malloc (4)
```

- address of the first byte is stored in the pointer.
- typecasting the void pointer returned by malloc if allocation was successful.
- allocates a size of 4 bytes.

NEW	MALLOC.
operator	function
calls constructor	doesn't call constructor.
implicit type casting	explicit type casting.
returns datatype	void returns void
std::bad_alloc on failure	NULL on failure
delete()	free()
No reallocation By compiler	realloc() size calculated manually.
Can be Overloaded	Can't be overloaded.

* Calloc.
- used to dynamically allocate multiple blocks of memory.

Differs from malloc as below:
→ calloc() needs two arguments instead of just one.

Syntax: void *calloc(size_t n, size_t size);

- n: number of blocks
- size: size of each block.

VIRAG

e.g.

```
int *ptr = (int *) calloc (10, sizeof (int));
```

Equivalent malloc call:

```
int *ptr = (int *) malloc (10 * sizeof (int));
```

→ Memory allocated by calloc is initialized to zero.

~ In case of malloc it is initialized with some garbage value.

~ Both return NULL when sufficient memory is not available in the heap.

calloc — clear allocation
malloc — memory allocation.

realloc.

- used to change the size of memory block w/o losing the old data.

Syntax: void *realloc (void *ptr, size_t newSize);

ptr → pointer to previously allocated memory
newSize → new size.

- on failure, returns NULL.

e.g.

```
int *ptr = (int *) malloc (sizeof (int));
ptr = (int *) realloc (ptr, 2 * sizeof (int));
```

- newly allocated bytes are uninitialized.

VIRAG

of(int));

eof(int));

eed to zero.

zed with some

ient memory is

ck w/o losing

- newSize){};

↑
new size.

);

eof(int));

Date / /20

* realloc
- It is used to release the dynamically allocated memory in heap.

Speed:
malloc realloc calloc

20-3-20

* Pointer to Constant Integer

const int *p;
≈ int const* p;

e.g. int a=10;
const int *pi;
pi = &a;
a = 30; ✓
*pi = 40; X

pi: [1] (2) a [10] (1)
*pi=30

* Constant Pointer.

int * const p = &var; ← Initialization should be done at the time of declaration only.

e.g.
int a=10; b=30;
int * const pi=&a;
*pi = 30; ✓
pi = &b; X

pi: [1] (3) → a [10] (1)
→ X b [30] (2)

* Constant Pointer to Constant Integer.

const int a=10;
int * const pi = &a;

pi [2] (1) a [10] (2)

Q. void func(const int *p){
    *p++;
    cout<<*p<<endl;
}
int main(){
    int arr=[1,2,3];
    func(&arr); → does not work.
                → To work, func(arr);
}

Q. void func(int * const p){
    *p=50;
}
int main(){
    int x=10;
    func(&x);
    cout<<x;
}

If we try in func,
int b=20;
p=&b; ✗
✗ - does not work

23-3-26

Exception Handling

Error: Any illegal operation performed by the user.

Error.
- Syntax Error (Compile Time Error) {semicolon nahi dala} → o/p not as expected, wrong reversal logic.
- Logical Error → division by 0 using fn. & file access & var. from other files & are not defined.
- Runtime Error
- Linking Error → c=a+b, a+b=c ✗
- Semantic Error → () func. { }

Exception
progr
- Thes
- mar
stru

try
try {

int main(
int
cout
cin
try {
if

e
}
catch
catch
catch

Date / /20

Exception: Unexpected event which disrupts the program flow.
- These can be handled & recovered from.
- manages runtime error in a ~~structured~~ structured way.

It is done using below keywords.

try → try{ throw(); }
catch → Built-in Types → C++ library for exception.
Standard Exceptions : exception
Custom Exception
throw

catch { → To handle the exception whatever is being thrown. }

catch (...) { } → Default catch block for different throw.

```
int main(){
    int n,d,e;
    cout<<"value of num. & deno.";
    cin >> n >> d;
    try {
        if(d==0)
            throw(0) / throw("Division by zero")
        e = n/d;
    }
    catch (int n){cout<<"Division by zero"}
    catch (const char* c) { ...}
    catch (...) { }
}
```

↳ default catch.

```
int main(){
  int n;
  cin>>n;
  int arr[n];
  for(int i=0;i<n;i++){
    cin>>arr[i];
  }
  int index;
  cout<<"Enter the index to search";
  cin>>index;
  try{
    if(index>=n)
      throw("Index out of bounds");
    cout<<arr[index]
  }
  catch(const char *e){
    cout<<e;
  }
  catch(...){
    cout<<"Index is not valid";
  }
}
```

* Standard Exception in C++
  Set of classes that represent common type of exceptions.
  e.g. std::bad_alloc, std::out_of_range.
  - These classes inherit 'exception' class in C++.
  - These classes have a virtual function what().
    with return type const char *.

```
virtual const char * what(){
  ...
}
```

→ try{

* Custom

class N

pr

public:

Nega

c

}

- noexcept:

- what() fn. is virtual, so that the developer can override the fn. to give a custom output/msg.
- what() is a fn. of 'exception' class → ?

e.g. (Standard Exception)

```
int main(){
    try{
        int *ptr = new int[10000000000];
        delete[] ptr;
    }
    catch(std::bad_alloc &ba){
        cout << "Exception in new" << ba.what();
    }
}
```

int *ptr = new int[size_t(-1)];

↳ Custom msg. ↳ Error msg.

→ try{} block needs atleast one catch{} block.

⊛ Custom Exception

```
class NegativeValueException: public exception{
    private:
    public: int value;
        NegativeValueException(int val){value=val};
    public:
        const char* what() const noexcept override{
            return "Negative Value Exception";
        }
};
```

- noexcept: tells the compiler that there won't be any exception in the function. You need not to worry.

- noexcept(false): This tells the compiler that there could be a exception in the function.

```
int main() {
    int arr = [1, 2, -1, 3];
    for(int n : arr){
        try {
            if (n<0) {
                throw NegativeValueException(n);
            }
            cout << n << endl;
        }
        catch (NegativeValueException &b) {
            cout << b.what();
        }
}
```

set_terminate.

- Customising terminating behaviour for uncaught exception.

Normally, if a exception is not ~~calle~~ caught then the compiler calls, from exception class.

unexpected()

↓

terminate()

↓

abort().

e.g. e.g.

void

int main

set_

try

}

ca

}

}

- set_terminate

function if

stack unwin

check for catch blo

void f1() {

void f2() {

f1

void f3()

f

int main(

f3

}

- goes on unwind

If not fou

ON TUE WED THU FRI SAT SUN

that there
ction.

(n);

)){

uncaught

t then

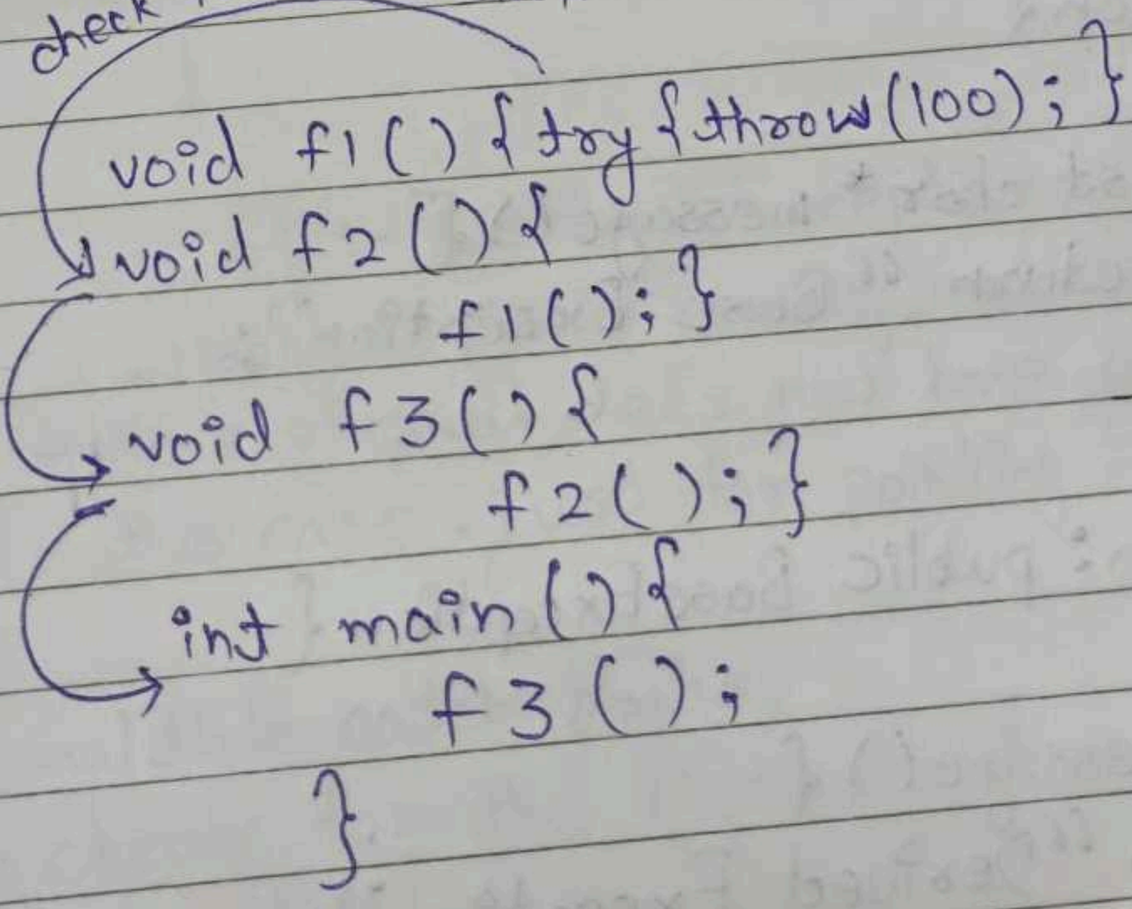
Date: / /20

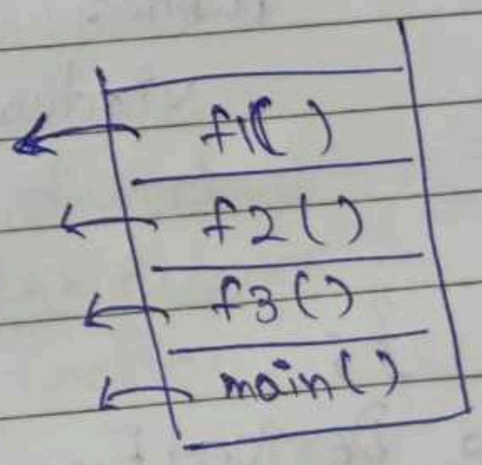
eg. e.g.

```
void myhandler(){
    cout << ----
    abort();
}

int main(){
    set_terminate(myhandler);
    try{
        throw(100);
    }
    catch(const char * a){
        cout << a << endl;
    }
}
```

- set_terminate expects a function which calls the function if a uncaught exception occurs.

stack unwinding.





- goes on unwinding, so that the caught block is found
If not found, the program aborts.

VIRAG

Date: / /20

- Stack unwinding happens when removes function call frames from the stack after its execution at runtime. It happens when:
  - Normally, when a function returns.
  - When exception is thrown but not caught immediately in some function. Control transfers to the matching function catch.

e.g.

```
void f1(){ throw(100);}
void f2(){ f1();}
void f3(){
    try {
        f2();
    }
    catch(int n){cout<<"Exception";}
}
int main(){
    f3();
}
```

```
class BaseException{
public:
    virtual const char* message(){
        return "Base Exception";
    }
};
class DerivedException: public BaseException{
public:
    const char* message(){
        return "Derived Exception";
    }
};
```


Date: / /

```
int ma
{
c
}
```

- It is b
  properti
  properti

- To get
  use pa

```
int main
    try
    cat
}
```

o/p:

{ ↔ → Wrong,

- Here, vir
  this case

For multiple ca
- checks for
  according l
  (or)
- gives the
  occurs fir

```
int main(){
    try { DerivedException(); }
    catch(BaseException b){
        cout << b.message();
    }
}
```

o/p :- Base Exception

- It is because, the base class object is created in catch & all the properties are copied into it as it is, remaining all properties of derived are neglected.
  — Concept of 'object slicing'.

- To get the desired output as we expect we should use pass by reference in catch block.

```
int main(){
    try { DerivedException(); }
    catch(BaseException &b){
        cout << b.message();
    }
}
```

o/p :- Derived Exception.

& ↔ → Wrong, it works normally, as it works if virtual is not present.
- Here, virtual does not have any significance in this case, just for pointing the function.

For multiple catch blocks,
- checks for the first-best-match & gives o/p accordingly.
(or)
- gives the one with in order, the one that comes occurs first serially.

Q. Write a Garage class that has a 'Car' that is having trouble with its 'Motor'. Use a function level try block in Garage class constructor to catch an exception (thrown from 'Motor' class) when it's 'Car' object is initialized. Throw a different exception from the body of 'Garage' constructor & catch it in main():

→

```
class Motor {
public:
    Motor() {
        throw(0);
    }
};

class Car {
public:
    Car() {
        Motor m;
    }
};

class Garage {
public:
    Garage() {
        try {
            Car c;
        }
        catch (int val) {
            cout << "Garage
                Exception" << endl;
            throw (-1);
        }
    }
};

int main() {
    try {
        Garage g;
    }
    catch (int n) {
        cout << "Main Exception" << endl;
    }
    return 0;
}
```

- used to av
→ provides a
within its
- Basi

```
namespace NS1
    void func(
        cout <
}
}

int main() {
    func()
}

using namespac
int main(
    func
}
```

- Nested namesp

```
namespace
    namespa
        v
                }
            }
        }
```

Date: 24/3/2026

Namespace

- used to avoid name conflicts.

→ provides a scope for identifiers (variables, functions) within its declarative region.

- Basically defines the scope.

```
namespace NS1 {
    void func() {
        cout << "In NS1";
    }
}
```

```
namespace NS2 {
    void func() {
        cout << "In NS2";
    }
}
```

```
int main() {
    func();
}
```

→ compiler doesn't understand from where this fn. implementation to be called.

```
using namespace NS1;
int main() {
    func();
}
```

→ calls from namespace NS1

```
int main() {
    NS1::func();
}
```

→ using scope resolution operator.

- Nested namespace.

```
namespace NS1 {
    int i;
    namespace NS2 {
        void func() {
            cout << "In NS1-NS2";
        }
    }
}
```

→ NS1::i;

NS1::NS2::func()

→ Discontinuous Namespace.
↳ e.g. ~~std~~ namespace std.
in iostream in vector

- namespace is spread across different files. At time of use all of them are combined.

⇒ Inline Namespace.

```
inline namespace NS1{
    inline namespace NS2{
        void func(){
            cout<<"NS2";
        }
    }

namespace NS3{
    void func(){
        cout<<"NS3";
    }
}
```

```
int main(){
    func();
    NS3::func();
}
```

"NS2"
"NS3"

NS1{
NS2{
NS3{
}}}func()

NS2::NS3:

NS1{
inline NS2{
inline NS3{
}}}func()

NS1::func() → no other provided func() is not present in namespace NS1

(*) Rules:
- Defined
nested n
- No semi
- Can have
merged i
- ~~No~~ No

e.g. namespa
cl
}

#include <io
using namesp

- How does cout
object of class
Similarly, for

- cout is a
- cin is a

→ cout & cin
classes, so we
class ostre

Date: / /20

(*) Rules:
- Defined in global, not inside main, can have nested namespaces.
- No semicolon after declaration.
- Can have different files of namespaces, at last merged into one.
- Namespace cannot be instantiated.

e.g. namespace NS1{
class A{
public:
int a;
};
}

```
#include <iostream>
using namespace std;
```

```
int main(){
NS1::A obj1;
} obj1.a;
```

→ works

```
namespace std{
    class istream{};
    class ostream{};
}
```

- How does cout output even if we do not create the object of class ostream which is in namespace std. & Similarly, for cin of class istream.

- cout is a object of ostream
- cin is a object of istream

→ cout & cin are global objects of their respective classes, so we need not create objects/instantiate class ostream & istream.

Date / /20

⇒ Templates

Blueprint — Works on different datatypes.

- Helps in Generic Programming.

```
void addInt(int a, int b)
{
    cout << a+b;
}

int main(){
    addInt(4,3);        // 7
    addFloat(4.3, 3.7); // 8
}
```

```
void addFloat(float a, float b)
{
    cout << a+b;
}
```

using template,

Function Template.

```
template <typename T>
void add(T a, T b){
    cout << a+b;
}
int main(){
    add(3,4);
    add(3.4, 4.6);
}
```

```
T add(T a, T b){
    return a+b;
}
```

At compile time, the compiler create's copies of the function as required, for each function call, one copy is created.

VIRAG

Date: / /20

Template Specialization.

- If one want to handle one or two datatype different & not as a template, it is called template specialization.
- Different behaviour for that specific DT other than a normal templates behaviour.

e.g. template <typename T>
void add(T a, T b){
  cout << a + b;
}

void add(int a, int b){
  cout << "This is int";
}

Template Specialization.

Function Template overloading

template < >
void add(int a, int b){
  cout << "This is template int";
}

~~default datatype~~

template<typename T=int>

e.g. template <typename T>

class Calculator{
public:
  void add(T a, T b){
    cout << a + b;
  }
};

class template can also be created using,
template<class T>

This is a class template.

int main(){
  Calculator <int> c1;
  c1.add(3,4); // 7
  Calculator <float> c2;
  c2.add(3.4, 4.6); // 8
}

Date: / /20

MON TUE WED THU FRI SAT SUN

typename :

```
template <typename T>
void fun ( T &x)
{
    static int i=10;
    cout << ++i;
}

int main () {
  int a=10;
  fun<int> (a);
  fun<int> (a);
  double b=1.1;
  fun<double> (b);
}
```

o/p:- 11
12
11

Imp:=

- for each fun template copy created, the static variable is shared only for the same kind of data types.
  for int, it shares a single static variable.
  for float, it shares a single static variable.
  !

Learn: static in non-static. → bcoz, its obj is not created.
nonstatic in static

- static is related to class
- non-static var members are related to object.

```
template <typename T>
class Calc {
public:
    static int i;
    Calc (){
        i++;
    }
    void count(){
        cout << i <<endl;
    }
};
template <typename T>
int Calc <T>::i=0;

int main () {
    Calc <int> obj,obj1;
    Calc <float> obj2,obj3,obj4;
    obj.count ();    //2
    obj2.count ();   //3
}
```

e.g.
te
vo
voi
int

If we
in

VIRAG

Date: / /20

static → no object → cannot use object things.
non-static → has object → can use both objects & class things.

e.g.
```
template <typename T>
void fun (T a, T b)
{
    cout << "Hello";
}
int main () {
    fun (10, 10.5);
}
```
→ Compile Time Error
- Typecasting is not performed.
- To make this work
template <typename T, typename U>

e.g.
```
template <typename T>
void fun (T &a){
    cout << "Reference";
}
void fun (T a){
    cout << "Value";
}
int main (){
    int x=10;
    fun (x);
}
```
→ Compile Time Error.
- ambiguous function fun.
- more than one instance of fun.

If we do,
```
int main(){
    int x=10;
    fun (&x);
}
```
o/p:- Value.

e.g. template <class T>
void fun (T a) {
    cout << sizeof(a);
}
int main () {
    int arr[5];
    fun(arr);
}

→ o/p: 8 (or) 4 (system dependant)

(array decay)

fun(arr)
|||
farr[0]
↳ 8

fun(&a)
|||
↳ 8

void fun(T &a)
{
}
fun(arr)
↳ 20

25-03-26

RAII

Resource Acquisition Is Initialization.

Resource: Memory, File Access, H/W Access

Principle: "Tie Resource lifetime to Object's lifetime"

int *ptr = new int();
delete ptr.

} Here, we need to explicitly ~~dest~~ delete the pointer.

↳ RAII says, there is no need to explicitly ~~dest~~ delete ptr.
ptr should be automatically ~~get~~ deleted as it goes out of scope.
i.e. Smart Pointers. for unique pointer ←

When an object is created → it acquires Resource
When an object is deleted → Resource should be deallocated automatically.
(goes out of scope)

→ Smart Pointers feature/concept applies RAII to dynamically allocated memory.

Smart Pointer

~~A class which wraps~~

A wrapper class that wraps ~~a~~ raw pointer which automatically manages dynamic memory allocation.
↳ lifetime of.

— This needs a header file called <memory>

Before C++17,
auto, unique, shared & weak.
After C++17,
auto was deprecated.

Advantages:
- Avoid memory leak
- Avoid dangling pointer
- Automatically manage dynamic memory.
- Avoid double deletion.
- Acquire memory in constructor
- Deallocate memory in destructor.
- Lightweight & memory efficient (No overhead of control block as for shared pointer)

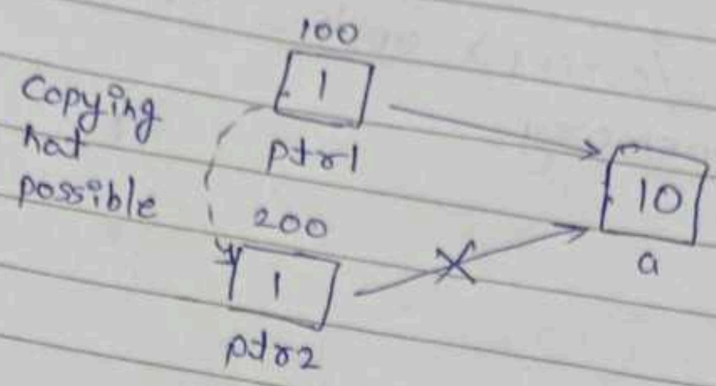
for unique pointer ←

VIRAG

Date / /20

- ✱ Unique Pointer
- ✱ Shared Pointer
- ✱ Weak Pointer

✱ Unique Pointer.

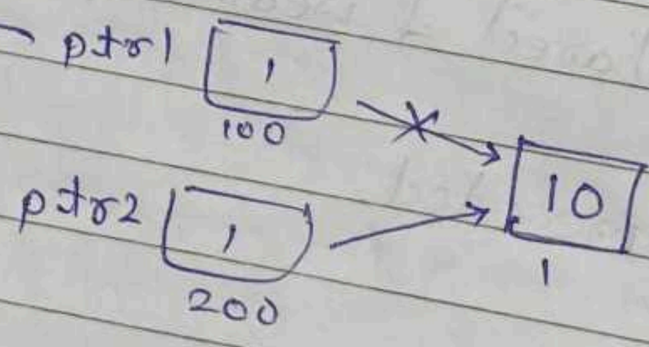


Copying not possible

single ownership.

```
std:: unique_ptr <int> ptr (new int (10));
std:: unique_ptr <int> ptr1 = make unique <int>100;
std:: unique_ptr <int> ptr 2 = ptr1 ;   ✗ this does not work
std:: unique-ptr <int> ptr 2 = std:: move (ptr1);
```

This works.



make-unique is a function of unique-ptr class in SP.

```
cout << *ptr1 ;   → 100
cout << ptr1 ;    → memory address.
```

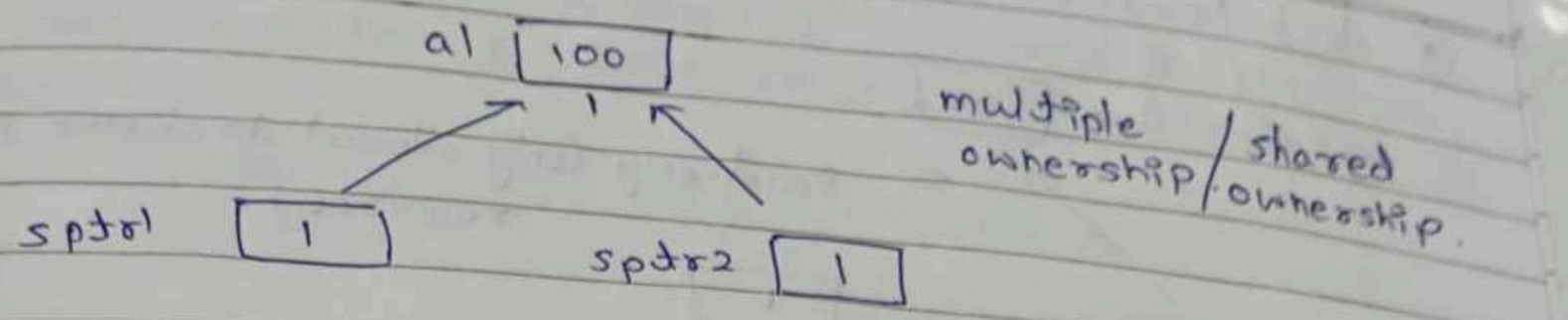
⇒ Only one unique pointer can own a resource at one time.

make-unique is preferred, since it has various overloaded fn. f feasibility to use.

VIRAG

Date: / /20

⊛ Shared Pointer.



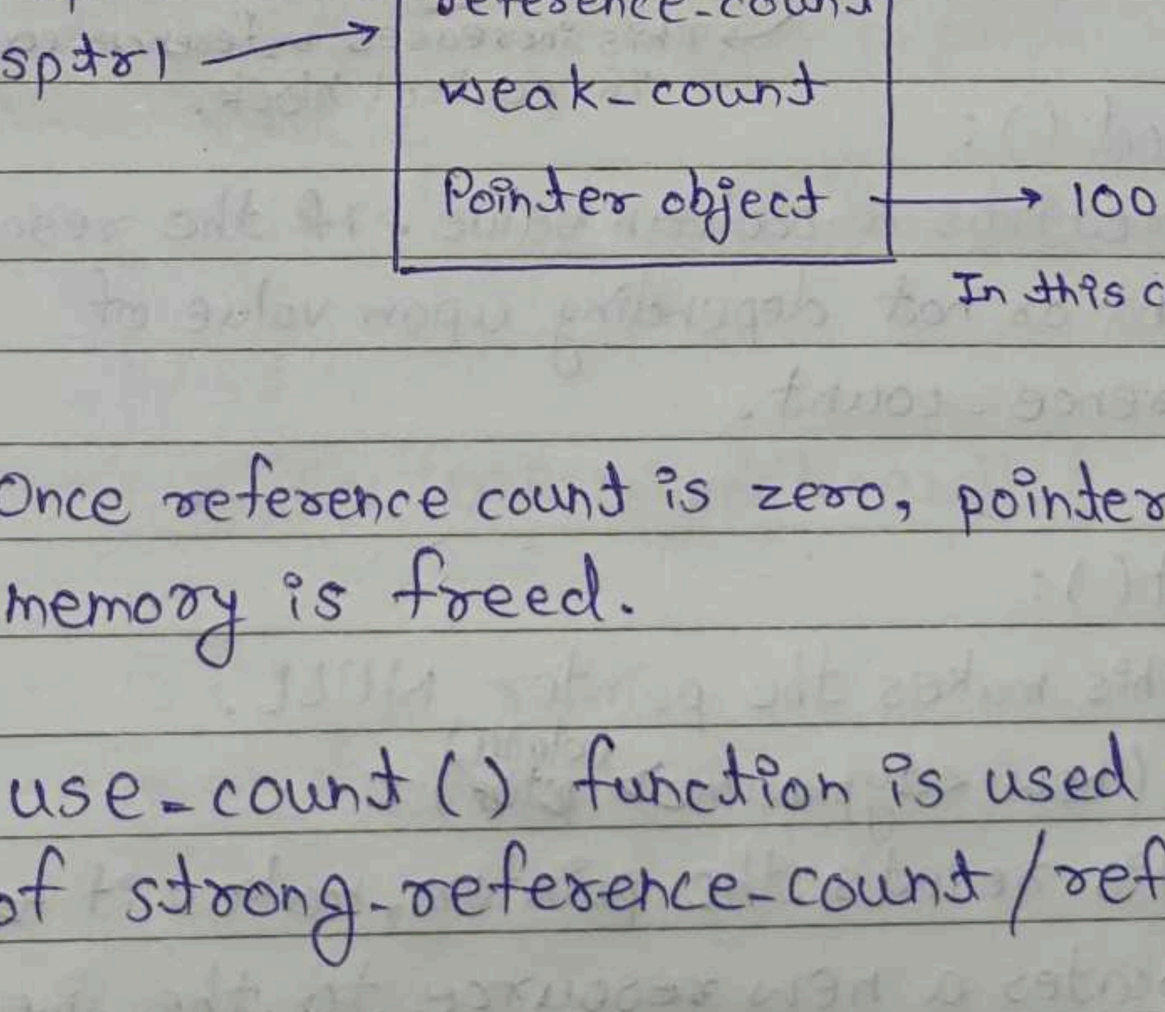

```
std:: shared_ptr<int> sptr = make_shared<int>(100);
std:: shared_ptr<int> sptr1 = sptr;
```
↳ This is allowed here.

— Reference Count / Strong Reference Count.

Any resource which is being shared by two or more pointers, the resource should not be deallocated if any pointer goes out of scope, so reference-count is maintained.

- ~~Increments on creation of a shared pointer~~
- Decrements on deletion of a shared pointer.

Shared Pointers have Control Block, → (A structure kind of thing were not sure)



— Once reference count is zero, pointer object is deallocated/ memory is freed.

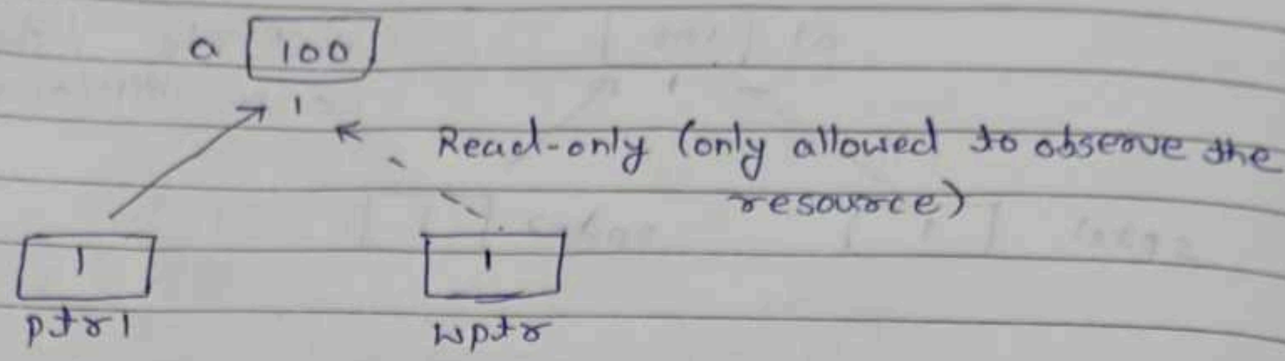
→ use_count() function is used to get the count of strong_reference_count / reference_count.

Advantages:

- shared.

Date / /20

⊛ Weak Pointer




```
std::shared_ptr<int> ptr1 = make_shared<int>(100);
std::weak_ptr wp = ptr1;
```

cout << *wp ⟶ cannot do this.

- to do this we need to use function lock().
- lock() returns/gives the resource/data.
- store that resource in a temporary shared pointer.
- returns NULL if the resource is not present.

```
std::shared_ptr<int> temppt = wp.lock();
```

↳ This increases reference-count in control block.

- expired():
  returns a boolean value, if the resource is present or not depending upon value of reference-count.

- reset():
  This makes the pointer NULL.
  - reset(new <typename>(data)):
    This resets the pointer, makes it NULL & allocates a new resource to the given pointer.

- It is a non-owning reference.

- Custom Deletor:

This is ~~if~~ so that if we want to perform some operation on deletion of a smart pointer, we can do it.

```
std:: ----<int, ---> ptr = ----;
```

↳ ~~operat~~ function in which the required operation is ~~to be~~ performed. being.

Cyclic Dependancy.

```
#include <memory>
class B;
class A {
    int x;
    shared_ptr<B> ptr B;
public:
    A(int val){
        ptr B= make_shared<B>(100);
        x=val;
        cout<<"CTR called"<<endl;
    }
    ~A(){
        cout<<"DTR called"<<endl;}

class B{
    int y; shared_ptr<A> ptr A;
public:
    B(int val){
        ptr A = make_shared<A>(200);
        x=val;
        cout<<"~~C~~CTR B called"<<endl;
    }
    ~B(){
        cout<<"DTR B called";
    }
```

Date: / /20

```
int main() {
    shared_ptr<A> a = make_shared<A>();
    shared_ptr<B> b = make_shared<B>();
    a->ptr B = b;
    b->ptr A = a;
    return 0;
}
```

using only a single class.

```
class R {
public:
    std::shared_ptr<R> m_ptr;
    R() { cout<< "Const." <<endl;}
    ~R() { cout<< "Dest." <<endl;}
}

int main() {
auto ptr = std::shared_ptr<R>();
    ptr->m_ptr = ptr;
}
```

- for single class, to work make the shared_ptr in class a weak_ptr.

data stor
linear &
sequentia
manner.

Fixed size Static
- Arra

Lin

single LL

Doubly LL NULL

Circular LL

Advantages
- Dyn

Date: 26/3/2026

Data Structure

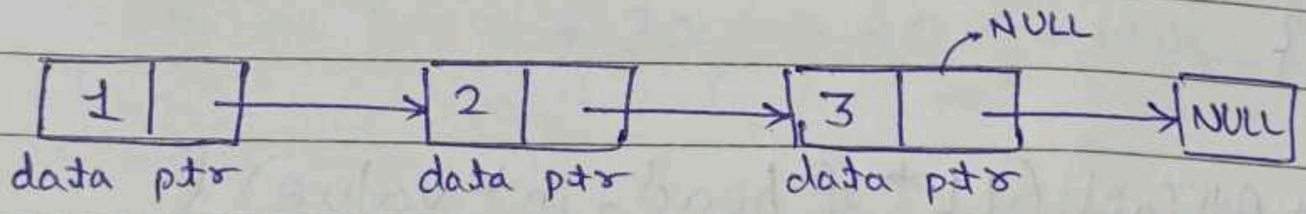
- organising data.

Processing, Retreving, Storing Data.

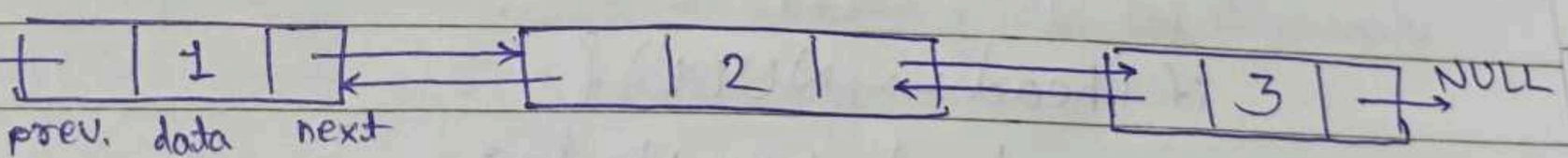
- Linear DS
  - data stored in linear & / or sequential manner.
  - Static DS (Fixed size)
    - Array
  - Dynamic DS (Variable size)
    - Vector
    - Linked List.
    - Stack
    - Queue.
- Non-Linear DS.
  - Not in continuous memory location.
  - Graph
  - Trees.

Linked List.

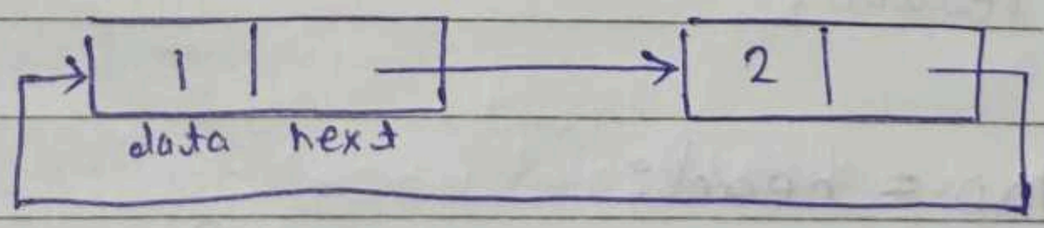
single LL



Doubly LL



Circular LL



Advantages of LL:

- Dynamic Size (Extend or Shrink).
- Better memory management.
- Insertion & Deletion are comparatively easier.

Disadvantages of LL:
- Searching is expensive.
- Additional memory for pointer.
- Accessing a node will take more time.

D. Usage of LL? (in Everyday Life)

```
class LL{
public:
    int data;
    LL *next;
};

LL(int value){
    data = value;
    next = NULL;
}

void insertAtHead(LL* &head, int value){
    LL* newNode = new LL(value);
    newNode->next = head;
    head = newNode;
}

void insertAtTail(LL* &head, int value){
    LL* newNode = new LL(value);
    if(head == NULL){
        head = newNode;
        return;
    }
    LL* temp = head;
    while(temp->next != NULL){
        temp = temp->next;
    }
    temp->next = newNode;
```

```
}
int ma
}
```

O/p:-

* Use refe
going to
pass the

```
void display (LL* head){
    while (head != NULL) {
        cout << head -> data << " -> ";
        head = head -> next;
    }
    cout << "NULL" << endl;
}

int main () {
    LL* head = NULL;
    insertAtHead (head, 20);
    insertAtHead (head, 10);
    display (head);
    insertAtTail (head, 30);
    insertAtTail (head, 40);
    display (head);
    return 0;
}
```

O/p :- 10 → 20 → NULL
10 → 20 → 30 → 40 → NULL

* Use reference for passing head only if you are going to modify the head. Else pass the head using pass by value.

Q- Manage 2 stacks in one array.

Q. Match the braces.
[({})]

Q.1. →

~~class Stack{~~
class Node {
    Node* next;
    int data;
}

class Stack {
    Node * top;
public:
    ~~Stack(){~~
    ~~Stack(int value){~~
        ~~data=value;~~
    ~~}~~
    Stack(){
        top=NULL;
    }

    void push(int value){
        Node * newNode = new Node(value);
        newNode->next = top;
        top = newNode;
    }

    int pop(){
        if(top == NULL) {cout<<"Stack Underflow"<<endl; return -1;}
        (return top->data)
        cout<<"Popped: "<<top->data<<endl;
        ~~top~~ Node * temp = top;
        top = top->next;
        delete temp;
    }

    Node(int value){
        data=value;
        next=NULL;
    }
}

what it
cal
re
what
sy
mem

Reference

```
int topOfStack() {
    if (top == NULL) {
        cout << "Stack is empty" << endl;
    }     return -1;
    return top->data;
}

bool isEmpty() {
    return top == NULL;
}

}
```

New Delete

what it does → allocates mem.
calls 'const' 'destr'
return type
what happens if it fails
syntax
mem. dia.

Runtime Polymorphism

↳ Method Overriding.
virtual → Base
Derived
Base ob → Derv.

Reference's Referent. Copy Constructor.

```
int a[1];
a[1] = 0;
```
→ Error.

Date: 30/3/2026

SOLID Principles.

S - Single Responsibility Principle
O - Open/Closed Principle
L - Liskov's Substitution Principle
I - Interface Segregation Principle
D - Dependency Inversion Principle.

1. S
- Give one responsibility to a class.
- Only one reason to change it.

2. O
- Class should be open for extension.
- Closed for modification.
  ↳ We should not modify the earlier written code.
  Modify the existing class unless it is necessary.

3. L (1987)
- We should be able to substitute the Derived class in a Base class.

Animal
↙ ↘
Dog ✓ Cat

Vehicle.
↙ ↘
Printer ✗ Computer

- Do not force a class to inherit a Interface/Abstract class which is not relevant.
- Do not inherit a Abstract class if it is not relevant.

4. I

4B
1B

- Static C

int a =
char c =
int *

Date: / /20

5. D.

- Always inherit from a Abstract class instead of a Concrete Class. (it is recommended, can't be followed always).
- Inheriting from concrete classes can expose the data.

Casting Operators.

- Converting one datatype to another.

```
int main (){
    int a=7, b=3;
    cout << (float) 7/3;
}
```

↳ C-style typecasting.

i) Implicit
ii) Explicit

4B → int i=300;
1B → char c=i;

float f=9.76;
int i=f;

0 ... ... 0 0 1 0 0 1 0 1 1 0 0

⇒ c = 44 = , (comma).

— Static Cast.

Compile Time Error are better than Runtime Errors.

It is dangerous

This can also work at sometimes.

```
int a=10;
char c='a';
int *q = (int*)&c;
```

→ This may compile but there will be undefined error at runtime.

```
int *r = static_cast<int*>(&c);
```

↳ This will give an error at compile time.

— Compile Time Checking

VIRAG

Date / /20

1. Dynamic Cast
Runtime checking on pointer & reference in inherited classes.

Downcasting
Base
↓
Derived.

Base *b = new Base();
Derived *d = b;
✗ (Not possible like above)

Base *b = new Derived();
Derived *d = b;
✓ (possible)
↳ Here using 'd' we can access the base class functions as well.

On success,
on pointer - Returns new-type.
on reference - Returns new-type.

On failure,
on pointer - gives NULL pointer.
on reference - bad cast exception.

Upcasting.
Derived
↓
Base
Base *b = &d
↳ No need for dynamic cast.

pdr

- Using co

int main(
const
Const
cout<<fun
}

Type ← mismatch

cout <<

int main(){
Const int v
Const int *
int

[vehicle*]
(vehicle*)

Vehicle
↙ ↘
Car Two wheeler
↓
gear()

using downcasting we can ensure that the vehicle pointer is of type car.

pointer & reference in

ecasting.

erived

se

*b = &d

to need for dynamic
ast.

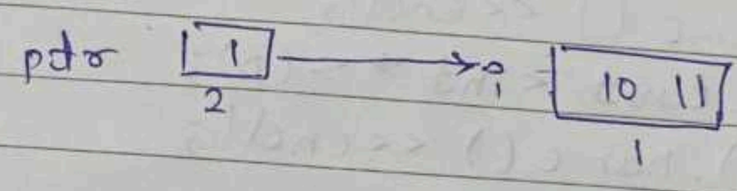
Date: / /20

- RTTI (Runtime Type Information) used by dynamic cast.
  - Is in V-TABLE.

- Const Cast.

```
const int i = 10;
int *ptr = const_cast<int*>(&i);
*ptr = 11;
cout << i << *ptr;
```

o/p:- 10 11 → Undefined Behaviour.



- Using const_cast helps us understand

```
int main(){
    const int val = 10;
    const int *ptr = &val;
    cout<<fun(ptr);
}
```

Type ← mismatch

```
cout << fun(ptr);
```

```
void fun(int *p){
    return (*p + 10);
}
```

This is in case, we are ensuring the function will not modify the value, it will just read the value & perform req. operations.

```
int main(){
    const int val = 10;
    const int *ptr = &val;
    int *ptr1 = const_cast<int*>(ptr);
    cout << fun(ptr1);
}
```

Date: / /20

class Student {
int x;
void fun() const {
// modify x
(const_cast<Student *>(this))->x = 5;
} }

- const_cast is also used to change the volatile features of a variable.

int main() {
int a = 10;
const volatile int *p = &a;
cout << typeid(a).name() << endl;
int * c1 = const_cast<int *>(p);
cout << typeid(c1).name() << endl;
}

o/p :- int const volatile * --ptr64
int * --ptr64.

— Standard Te

Containers:
- stores dat
processed.
- Vector, Sta

Sequence Container

- Linear Way
- Vector, Array.

Algorithms:
- Predefined Fu

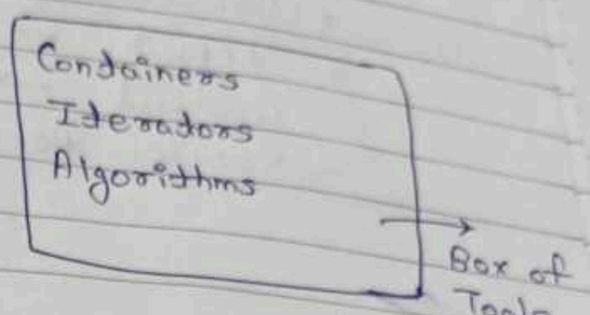
Iterators:
- Traverse thro

vector<int

Date: 31/03/2026

STL Vector

— Standard Template Library



Containers:
- stores data which is to be processed.
- Vector, Stack, List, Map, Set, ...

Types of Containers

Sequence Container.	Associative Containers	Unordered Associative Containers	Container Adaptors.
- Linear Way - Vector, Array.	- Unique & Sorted. Set, MultiSet, Map, Multimap	- Unsorted. Unordered Map, Unordered MultiMap.	- Adds some restrictions on how we access the data. - Stack, Queue

Algorithms:
- Predefined Functions.

Iterators:
- Traverse through the container.

vector<int>::iterator it

'it' is object of the above type.

⇒ Common way of traversing through the container without exposing the internal implementation.

Lambda Function ?

Date: / /20

Types of Iterators :
- Forward → Forward → for List
- Bidirectional → Forward/Backward → used for Doubly Linked List
- Input → reading
- Output → writing
- Random Access. → Random

⇒ Vector.
- Implemented as Dynamic Array.
- It can grow or shrink.

vector <datatype> vector-name;

vector <int> v;

v.push-back ();
pop-back ();
erase (); → with index.
emplace-back ();
clear ();
insert ();
empty ();
size ();
capacity ();
resize ();

#include <vector>

vector <int> v;

sizeof (vector)
↳ calculates size of a vector object, not the actual vector.

vector object has:
- 3 pointers.
- pointer to the vector/data
- The current size
- The current capa...

Types
→ Cr

→ Stru
C
str

• Common Pl
• Best Pra
• Efficient

→ We can cre
but this

Singleton.
Only one

- Private
- Static

ON TUE WED THU FRI SAT SUN

used for
Doubly Linked List

Date: / /20

MON TUE WED THU FRI SAT SUN

Design Patterns.

- Represents best practices used by experienced software engineers.
- Gang of Four documented it.
- Solutions to diff. problems that software developers face.

Types of Design Patterns

→ Creational
Creation of objects → Singleton

→ Behavioural → Observer.
How classes & objects communicate.

→ Structural
Classes & objects are composed to create entire structure.

- Common Platform for Developers.
- Best Practices
- Efficient.

→ We can create a singleton class using other methods as well but this is a standard method.

Singleton.
Only one object is to be created of the object.

of a
, not
tor.
r/data
y

⇒ - Private constructor.
- Static Instance
- Static Member Function.

VIRAG

```
class Singleton {
    Singleton () { cout << "Singleton CTR"; }
    static Singleton * sinstance;
public:
    static Singleton * getInstance () {
        if (! sinstance)
            sinstance = new Singleton ();
        return sinstance;
    }
};

Singleton * Singleton :: sinstance = NULL;

int main () {
    Singleton *s1 = Singleton :: getInstance ();
    Singleton *s2 = Singleton :: getInstance ();
    cout << s1 == s2;
    Singleton *s3 = new Singleton ();
}
```

↳ We cannot do this, constructor is private

- To avoid making copies of the singleton object, one should delete the copy constructor & also the overloaded assignment operator should be deleted.
- delete is a keyword in modern C++.

```
Singleton (const Singleton & s) = delete;
```

Date:

Obs

Data Member

class

Relis

1. Problem

→ Section

→ Section

- 4 m

2. Logical

Date: / /20

Observer.

Subject

Data Member → Status —Notified→ Observer.

Observer 1
Observer 2
Observer 3
⋮

1 → Many.

class Subject
~~Prot~~ ↓
int status;
vector Observer
addNewObserver()
RemoveObserver()
SetStatus()
Notify()

→ Base class
class Observer.
↓
update() = 0

Obs 1 Obs 2 Obs 3.

↳ Inherited from observer. should implement the update function.

Relis Score.

1. Problem Solving
   → Section - 1 (Puzzle).
   - 2 M - 5 Q
   → Section - 2 (Puzzle)
   - 4 mark - 5 Q

2. Logical Reasoning.
   → Section - 1 — 9 Points
   - 2 M - 5 Q
   → Section - 2 — 10 Points

3. CS Fundamentels
   2 M - 5 Q
   (MCQ)

4. DSA. — 6 points
   2 M - 5 Q
   (MCQ)

5. Database Fundamentals.
   2 M - 2 Q

6. Program (Coding)
   20 M

VIRAG